# 大数据获取与采集

BIG DATA





# 大数据获取与采集

CONTENTS


01 分布式爬虫
Distributed Web Crawler

02 日志采集
Log Collection

03 物联网数据采集
IoT Data Collection

04 扩展的数据采集
Extended Data Collection

重庆大学 CHONGQING UNIVERSITY

## 1.1 爬虫概念与基本原理

### 什么是网络爬虫

**网络爬虫**（Web Crawler/Spider）是一种按照一定规则，自动抓取万维网信息的程序或脚本。
它模拟浏览器发送HTTP请求，获取网页HTML内容，解析提取所需数据，并按链接继续遍历，实现互联网数据的自动化采集。
爬虫是大数据获取的核心技术手段，为搜索引擎、数据分析、AI训练提供海量原始数据。

### 爬虫工作流程




### 爬虫的核心技术要点

**HTTP协议**：GET/POST请求、请求头（User-Agent、Cookie、Referer）、响应状态码（200/301/403/404）
**网页解析**：XPath路径表达式、CSS选择器、正则表达式、JSON解析（API接口数据）
**遍历策略**：深度优先DFS、广度优先BFS、最佳优先（基于优先级），URL去重（布隆过滤器、哈希集合）

## 1.2 爬虫分类与技术演进

### 爬虫按规模与架构分类

**通用爬虫**

**代表**：搜索引擎爬虫（Googlebot、Baiduspider）
**特点**：大规模、全量抓取、高并发
**目标**：尽可能多的网页，建立索引

**聚焦爬虫**

**代表**：主题爬虫、垂直领域爬虫
**特点**：选择性抓取、主题相关
**目标**：特定领域/主题的数据

**增量式爬虫**

**特点**：只抓取更新/新增页面
**优势**：节省带宽、实时性好
**策略**：基于更新频率预测

**分布式爬虫**

**代表**：Scrapy-Redis、分布式爬虫集群
**特点**：多节点协作、共享URL队列
**目标**：海量数据、高吞吐采集

### 爬虫技术演进历程

- **2000s初：单线程脚本**
  Python urllib + 正则表达式，简单页面抓取
- **2000s中：多线程框架**
  BeautifulSoup、lxml解析，多线程并发
- **2010s：Scrapy框架**
  异步Twisted引擎、中间件、管道，企业级爬虫
- **2015s至今：分布式+智能**
  Scrapy-Redis分布式、Selenium/Playwright渲染、AI识别

**关键趋势**：从单机到分布式、从静态到动态渲染、从规则到智能识别、从批量到实时流采集。分布式爬虫通过多节点协作共享任务队列，实现海量数据的高吞吐采集，是大数据时代的核心技术。

## 1.3 Scrapy框架架构与核心组件

*Scrapy是基于Twisted异步网络框架的开源爬虫框架，采用组件化设计，支持高并发、可扩展的爬虫开发*




### 中间件机制

**下载器中间件**（Downloader Middleware）：
位于引擎和下载器之间，处理Request/Response
常用：User-Agent轮换、代理IP、重试、Cookie管理
**爬虫中间件**（Spider Middleware）：
位于引擎和爬虫之间，处理Response和Item
常用：异常处理、URL过滤、深度控制

### Scrapy核心优势

**异步高性能**：基于Twisted事件驱动，单进程支持数千并发
**组件化设计**：引擎、调度器、下载器、爬虫、管道解耦，易扩展
**内置功能**：自动去重、重试、限速、日志、Telnet控制台
**生态丰富**：Scrapy-Redis分布式、Splash渲染、众多插件

## 1.4 Scrapy-Redis分布式架构原理

*Scrapy-Redis通过Redis实现多爬虫节点共享URL队列和去重集合，突破单机性能瓶颈，实现水平扩展*




### Scrapy-Redis核心机制

**共享调度器**：Redis List作为URL队列，所有节点LPOP/RPOP消费，实现任务分发；Redis Set作为指纹集合，全局去重避免重复抓取
**两种运行模式**：Master-Slave模式（Master推送URL，Slave消费）和Crawl模式（所有节点平等消费，通过start_urls初始化）
**断点续爬**：Redis持久化（RDB/AOF）保证队列不丢失，节点重启后从断点继续，支持增量爬取

# 1.5 Scrapy-Redis实操：环境搭建与项目结构

## Step 1: 环境安装

```
# 安装Scrapy和Scrapy-Redis
pip install scrapy scrapy-redis
# 安装Redis
brew install redis # macOS
sudo apt install redis-server # Ubuntu
# 启动Redis服务
redis-server /etc/redis/redis.conf
```

## Step 2: 创建Scrapy项目

```
scrapy startproject myspider
cd myspider
scrapy genspider example example.com
```

## 项目目录结构

```
myspider/
├── scrapy.cfg # 项目配置
└── myspider/
    ├── __init__.py
    ├── items.py # 数据结构定义
    ├── middlewares.py # 中间件
    ├── pipelines.py # 数据管道
    ├── settings.py # 配置文件
    └── spiders/
        ├── __init__.py
        └── example.py # 爬虫脚本
```

**settings.py 关键配置（Scrapy-Redis）**

```
SCHEDULER = "scrapy_redis.scheduler.Scheduler" # Redis调度器
DUPEFILTER_CLASS = "scrapy_redis.dupefilter.RFPDupeFilter" # Redis去重
REDIS_URL = "redis://localhost:6379" # Redis连接
ITEM_PIPELINES = {"scrapy_redis.pipelines.RedisPipeline": 300} # 数据入Redis
```

# 1.6 Scrapy-Redis实操：核心代码实现

## items.py - 数据结构定义

```
import scrapy
class BookItem(scrapy.Item):
title = scrapy.Field() # 书名
price = scrapy.Field() # 价格
author = scrapy.Field() # 作者
rating = scrapy.Field() # 评分
url = scrapy.Field() # 详情页URL
```

## 启动分布式爬虫

```
# 节点1、2、3分别启动（代码相同）
scrapy crawl book
# 推送初始URL到Redis队列
redis-cli lpush book:start_urls \
http://books.toscrape.com/
```

## spider.py - 分布式爬虫核心

```
from scrapy_redis.spiders import RedisSpider
from ..items import BookItem
class BookSpider(RedisSpider):
name = 'book'
redis_key = 'book:start_urls'
def parse(self, response):
# 解析列表页，提取详情链接
books = response.css('article.product_pod')
for book in books:
item = BookItem()
item['title'] = book.css(
'h3 a::attr(title)').get()
item['price'] = book.css(
'.price_color::text').get()
yield item
# 翻页，新URL自动入Redis队列
next_page = response.css(
'.next a::attr(href)').get()
if next_page:
yield response.follow(
next_page, self.parse)
```

**代码要点**：继承RedisSpider而非scrapy.Spider；定义redis_key指定Redis队列名；无需start_urls，初始URL通过redis-cli lpush推送；所有节点运行相同代码，自动从共享队列消费URL；parse方法中yield item和yield Request，新URL自动进入Redis队列实现分布式。

**运行流程**：启动多个爬虫节点（等待状态）→ 推送初始URL到Redis → 某个节点获取URL开始爬取 → 发现的新URL入队 → 所有节点竞争消费 → 数据通过RedisPipeline写入Redis → 可再由消费者程序写入MySQL/MongoDB。

## 1.7 反爬机制与应对策略

| 常见反爬机制 | 应对策略 | 技术实现 |
| --- | --- | --- |
| User-Agent检测<br>识别默认爬虫UA，拒绝访问 | UA轮换<br>随机切换浏览器User-Agent | USER_AGENTS = [列表]<br>RandomUserAgentMiddleware |
| IP频率限制<br>单IP请求过快触发封禁 | 代理IP池+限速<br>多IP轮换，控制请求间隔 | HttpProxyMiddleware<br>DOWNLOAD_DELAY = 2 |
| 验证码<br>图形/滑块/点选验证码拦截 | 打码平台+AI识别<br>OCR识别、深度学习验证码破解 | ddddocr / 超级鹰<br>Selenium + 滑块模拟 |
| 动态渲染<br>JS渲染页面，静态HTML无数据 | 无头浏览器+API逆向<br>渲染JS或直接抓后端API接口 | Playwright / Selenium<br>Chrome DevTools抓包 |

其他反爬手段：Cookie验证、Referer检查、Token签名、字体加密、CSS偏移混淆、蜜罐链接、行为分析（鼠标轨迹/停留时间）

应对原则：优先分析API接口（最高效）→ 其次用无头浏览器 → 最后考虑验证码破解；遵守robots.txt协议，控制爬取频率，避免对目标网站造成压力

重庆大学 CHONGQING UNIVERSITY

## 1.8 实战案例：电商商品数据采集

*案例背景：某电商数据分析公司需要采集主流电商平台商品信息，用于价格监控、竞品分析和市场趋势研究*

**需求分析**

采集字段：商品名、价格、销量、评分、评论数、店铺名
目标平台：淘宝、京东、拼多多
数据量：日均100万+商品
更新频率：每6小时全量更新

**架构设计**

Scrapy-Redis分布式集群
5台爬虫节点（每平台独立Spider）
Redis共享队列+去重
代理IP池（1000+IP）

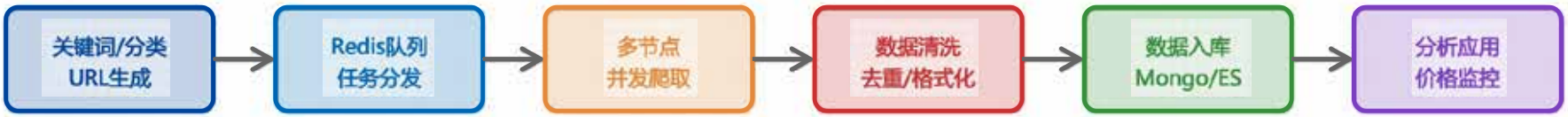
**关键技术**

API逆向：抓包分析移动端接口
签名破解：参数加密算法还原
Playwright：处理JS渲染页面
验证码：AI识别+人工打码

**数据存储**

MongoDB存原始数据
MySQL存结构化数据
Elasticsearch全文检索
Kafka实时数据流

**数据采集流程图**




**项目成果**：日均采集120万+商品数据，覆盖3大电商平台50+品类，数据延迟小于6小时，支持价格异动实时告警，为客户节省人工调研成本80%
**技术难点**：平台反爬持续升级（签名算法每2周更新）、代理IP质量不稳定、大促期间页面结构变化、移动端API参数加密（AES/RSA混合加密）

## 1.9 爬虫法律合规与伦理规范

### 相关法律法规

**《网络安全法》**（2017年施行）：禁止非法侵入他人网络、干扰网络正常功能、窃取网络数据

**《数据安全法》**（2021年施行）：规范数据处理活动，保障数据安全，促进数据开发利用

**《个人信息保护法》**（2021年施行）：处理个人信息需取得同意，禁止过度采集，保障个人信息权益

**《刑法》第285条**：非法侵入计算机信息系统罪、非法获取计算机信息系统数据罪，最高可判7年

### 合规爬虫准则

**遵守robots.txt**：检查网站robots协议，尊重Disallow规则，不爬取禁止的路径

**控制访问频率**：设置合理的下载延迟（DOWNLOAD_DELAY），避免对目标服务器造成压力

**保护个人隐私**：不采集个人敏感信息（身份证、手机号等），匿名化处理用户数据

**数据合理使用**：采集数据用于学习研究或商业授权用途，不传播、不转卖、不用于非法目的

**典型案例警示**：2019年"爬虫第一案"——某公司非法抓取淘宝数据，构成非法获取计算机信息系统数据罪，3名被告人被判处有期徒刑；2021年某招聘平台爬虫案，抓取500万条简历数据，主犯获刑3年。技术无罪，但使用必须合法合规。



# 大数据获取与采集

CONTENTS


01 分布式爬虫
Distributed Web Crawler

02 日志采集
Log Collection

03 物联网数据采集
IoT Data Collection

04 扩展的数据采集
Extended Data Collection

# 2.1 日志概念、类型与价值

## 什么是日志

**日志**（Log）是系统运行过程中产生的事件记录，包含时间戳、事件类型、严重级别、详细描述等信息。
日志是系统的"黑匣子"，记录了系统运行的每一个细节，是问题排查、性能分析、安全审计和业务洞察的重要数据来源。

## 日志的核心价值

**故障排查**
快速定位系统异常
根因分析

**安全审计**
入侵检测
合规审计

**性能监控**
系统性能指标
容量规划

**业务分析**
用户行为分析
商业智能

## 常见日志类型

**Web服务器日志**
Nginx/Apache access.log
访问记录、状态码、耗时
格式：Combined/Common

**应用程序日志**
Java/PHP/Python应用日志
Log4j/Logback框架
级别：DEBUG/INFO/WARN/ERROR

**系统日志**
Linux /var/log/messages
syslog、systemd journal
内核、服务、登录记录

**安全日志**
防火墙、IDS/IPS日志
审计日志、登录日志
合规要求：等保2.0

## 2.2 日志采集架构：ELK与EFK

*ELK = Elasticsearch + Logstash + Kibana | EFK = Elasticsearch + Filebeat + Kibana（Filebeat替代Logstash做采集端）*



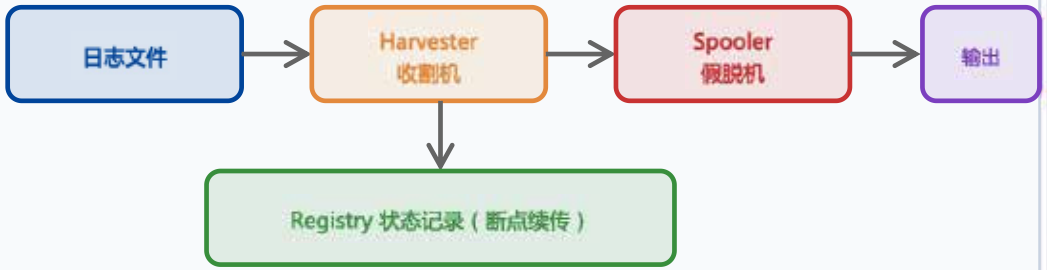

### ELK vs EFK 对比

**ELK架构**：Logstash做采集+处理
优点：功能强大，支持复杂过滤转换
缺点：Logstash资源消耗大（JVM），不适合大量边缘节点部署
适用：数据量中等、需要复杂ETL的场景

**EFK架构（推荐）**：Filebeat采集 + Logstash处理
优点：Filebeat轻量（Go编写，内存<50MB），适合大规模部署
缺点：Filebeat仅采集，复杂处理仍需Logstash
适用：大规模生产环境，主流推荐方案

## 2.3 Filebeat：轻量级日志采集器

### Filebeat工作原理




Harvester：负责逐行读取单个文件内容，每个文件一个Harvester
Spooler：汇总Harvester数据，批量发送到输出端
Registry：记录每个文件的读取偏移量，保证至少一次投递

### filebeat.yml 核心配置

```
# 输入配置
filebeat.inputs:
- type: log
enabled: true
paths:
- /var/log/nginx/*.log
fields:
log_type: nginx_access
multiline.pattern: '^\['
multiline.negate: true
multiline.match: after

# 输出到Kafka
output.kafka:
hosts: ["kafka1:9092","kafka2:9092"]
topic: '%{[fields.log_type]}'
partition.round_robin:
reachable_only: true
required_acks: 1
compression: gzip

# 输出到Logstash(可选)
# output.logstash:
# hosts: ["logstash:5044"]
```

# 2.4 Logstash：日志处理与转换引擎

*Logstash是开源的数据收集引擎，支持实时数据管道，具备200+插件，可从多种来源采集、转换并发送数据*




## logstash.conf 配置示例（Nginx日志解析）

```
input {
kafka {
bootstrap_servers => "kafka1:9092,kafka2:9092"
topics => ["nginx_access"]
consumer_threads => 4
group_id => "logstash_nginx"
}
}
filter {
grok { match => { "message" => "%{COMBINEDAPACHELOG}" } }
date { match => ["timestamp", "dd/MMM/yyyy:HH:mm:ss Z"] target => "@timestamp" }
geoip { source => "clientip" target => "geoip" }
mutate { convert => { "bytes" => "integer" "response" => "integer" } remove_field => ["message","timestamp"] }
useragent { source => "agent" target => "useragent" }
}
output {
elasticsearch { hosts => ["es1:9200","es2:9200"] index => "nginx-access-%{+YYYY.MM.dd}" }
}
```

# 2.5 Flume：分布式日志采集系统

## Flume架构与核心组件




Source：avro/thrift/exec/spooldir/syslog/http
Channel：memory(快但易丢)/file(可靠)/jdbc
Sink：hdfs/hive/kafka/hbase/logger/avro
Agent：一个JVM进程，包含Source+Channel+Sink

## Flume拓扑结构



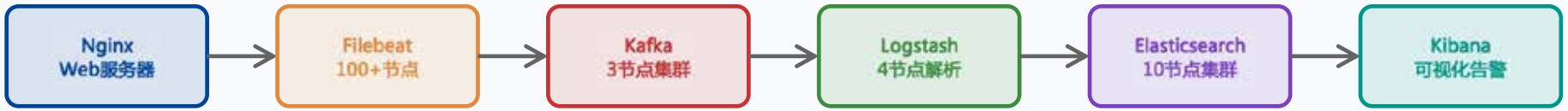

**多级聚合**：第一层Agent采集各服务器日志，第二层Aggregator聚合后写入HDFS
**负载均衡**：Source支持负载均衡，Sink组支持故障转移

## Flume vs Filebeat 对比

| 对比维度 | Apache Flume | Elastic Filebeat |
|---|---|---|
| 开发语言 | Java（JVM，内存消耗大） | Go（轻量，内存<50MB） |
| 适用场景 | 大数据生态，HDFS/Hive/Kafka | ELK生态，Elasticsearch/Logstash |
| 数据可靠性 | File Channel保证不丢数据 | Registry记录偏移量，至少一次 |
| 功能丰富度 | Source/Channel/Sink插件丰富 | 采集为主，处理需Logstash |

# 2.6 实战案例：Nginx日志采集与分析

*案例背景：某电商平台日均PV 5000万+，需要实时监控网站访问情况、异常告警和用户行为分析*




完整链路：Nginx产生日志 → Filebeat采集 → Kafka缓冲 → Logstash解析(grok+geoip+useragent) → Elasticsearch存储 → Kibana可视化

## Nginx Combined日志格式示例

```
192.168.1.100 - - [05/Sep/2026:10:23:45 +0800]
"GET /api/products?id=123 HTTP/1.1" 200 1532
"https://www.example.com/" "Mozilla/5.0 (Macintosh...)"
"0.032" "upstream_response_time=0.028"
```

## 解析后结构化字段

clientip: 192.168.1.100 | timestamp: 2026-09-05T10:23:45
method: GET | url: /api/products | status: 200
bytes: 1532 | referer: www.example.com
geoip: 重庆/中国 | useragent: Chrome/Mac

**项目成果**：日志处理延迟<30秒，支持实时监控PV/UV/响应时间/错误率；5xx错误超过阈值自动告警；按天建索引，保留30天热数据+90天温数据；Kibana仪表盘支持访问趋势、地域分布、TOP页面、浏览器占比等20+可视化图表

**关键优化**：Kafka分区按log_type划分提高并行度；Logstash使用pipeline.workers=8提升吞吐；Elasticsearch使用ILM（索引生命周期管理）自动滚动和删除

## 2.7 日志采集最佳实践与常见问题

### 最佳实践

1. **统一日志格式**：使用JSON格式输出日志，避免正则解析的脆弱性

2. **加入消息队列**：采集端与处理端之间加Kafka，解耦+削峰+缓冲

3. **索引生命周期管理**：ES使用ILM，热温冷架构，自动滚动+删除

4. **监控采集自身**：监控Filebeat/Logstash/ES的健康状态和积压量

### 常见问题与解决方案

**日志丢失**：Filebeat用registry保证断点续传；Kafka设置acks=all

**解析失败**：grok匹配失败的日志打标签，单独索引排查，不丢弃

**磁盘爆满**：ES设置索引大小上限+自动删除；Logstash持久化队列

**时间错乱**：统一使用UTC存储，展示时转换；NTP同步所有服务器

**选型建议**：小规模（<10台服务器）直接Filebeat→Elasticsearch；中大规模必须加Kafka做缓冲；大数据生态（Hadoop/Hive）选Flume；ELK生态选Filebeat+Logstash。日志量预估：每条日志平均200字节，日均1亿条约20GB/天，ES存储放大系数约2.5倍，需预留50GB/天。



# 大数据获取与采集

CONTENTS


01 分布式爬虫
Distributed Web Crawler

02 日志采集
Log Collection

03 物联网数据采集
IoT Data Collection

04 扩展的数据采集
Extended Data Collection

## 3.1 IoT概念与四层体系架构

### 什么是物联网（IoT）

**物联网**（Internet of Things，IoT）是通过传感器、射频识别（RFID）、全球定位系统等信息传感设备，按约定协议，把任何物品与互联网连接，进行信息交换和通信，以实现智能化识别、定位、跟踪、监控和管理的网络。

核心特征：全面感知、可靠传输、智能处理。

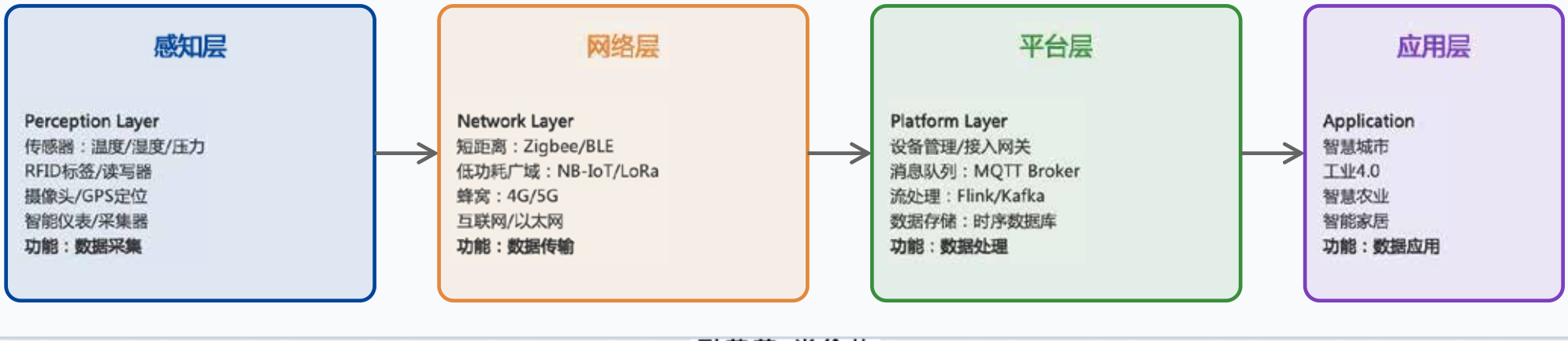
### IoT数据规模与增长

**连接设备数**：2025年全球IoT设备预计超300亿台

**数据量**：IoT数据年增长率超25%，2025年达79ZB

**应用领域**：智慧城市、工业4.0、智慧农业、智能家居、车联网

### IoT四层体系架构



## 3.2 IoT数据特点与采集挑战

### IoT数据六大特点

**海量性 Volume**
数十亿设备持续产生数据，单工厂日均TB级

**实时性 Velocity**
毫秒级采集频率，要求低延迟处理和响应

**多样性 Variety**
结构化/半结构化/非结构化，多源异构数据

**时序性 Time-series**
带时间戳的连续数据流，强时间相关性

**低价值密度 Value**
单条数据价值低，需聚合分析才能提取价值

**不完整性 Veracity**
噪声、缺失、异常值普遍，数据质量参差不齐

### IoT数据采集面临的挑战

**设备异构性**
不同厂商设备协议不同
Modbus/OPC-UA/MQTT
需统一接入网关适配

**网络不稳定**
低功耗广域网带宽有限
设备频繁上下线
需断点续传和本地缓存

**数据质量差**
传感器漂移和噪声
数据缺失和重复
需清洗和异常检测

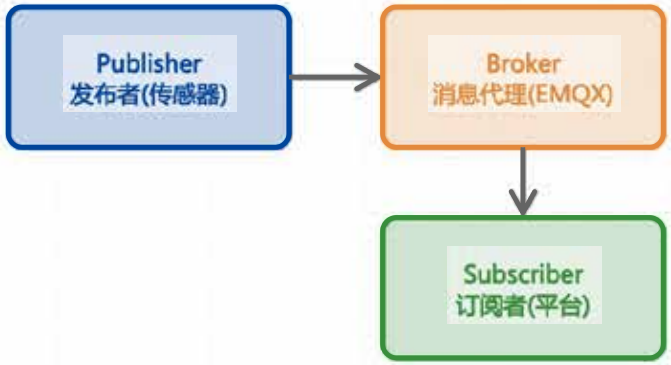
**安全与隐私**
设备身份认证
数据传输加密
边缘侧数据脱敏

# 3.3 MQTT协议：IoT消息传输标准

## MQTT协议概述

MQTT（Message Queuing Telemetry Transport，消息队列遥测传输）是OASIS标准的轻量级发布/订阅消息协议，基于TCP/IP，专为低带宽、不稳定网络环境设计。
最小报文仅2字节，是IoT领域最主流的通信协议。

## 发布/订阅架构




*发布者与订阅者解耦，通过Topic主题匹配，Broker负责消息路由*

## QoS服务质量等级与核心特性

| QoS等级 | 名称 | 语义 | 适用场景 |
|---|---|---|---|
| QoS 0 | 至多一次 | 消息最多送达一次，可能丢失，不确认 | 传感器周期上报，丢几条无所谓 |
| QoS 1 | 至少一次 | 消息至少送达一次，可能重复，需确认 | 命令下发、告警通知，不能丢 |
| QoS 2 | 恰好一次 | 消息恰好送达一次，四次握手保证 | 计费、交易等不能重复的场景 |

# 3.4 CoAP与LwM2M：受限设备协议

## CoAP协议

CoAP（Constrained Application Protocol，受限应用协议）是专为资源受限设备设计的Web传输协议，RFC 7252标准。

特点：

基于UDP，报文头仅4字节

RESTful架构，支持GET/POST/PUT/DELETE

支持订阅/通知（Observe模式）

内置资源发现机制

DTLS加密（类似TLS for UDP）

适用：MCU级低功耗设备、NB-IoT终端

## LwM2M协议

LwM2M（Lightweight M2M，轻量级M2M）是OMA定义的设备管理协议，基于CoAP传输。

特点：

定义了标准对象模型（设备/固件/位置）

支持设备注册、固件升级、参数配置

资源模型：Object/Object Instance/Resource

支持Bootstrap引导流程

电信运营商广泛采用（NB-IoT标配）

适用：大规模IoT设备管理、运营商物联网

## IoT通信协议对比

| 协议 | 传输层 | 报文开销 | 模式 | 功耗 | 典型应用 |
|---|---|---|---|---|---|
| MQTT | TCP | 最小2字节 | 发布/订阅 | 低 | 智能家居、工业IoT、消息推送 |
| CoAP | UDP | 最小4字节 | 请求/响应 | 极低 | NB-IoT终端、传感器节点、智能电表 |
| LwM2M | CoAP/UDP | 小 | 设备管理 | 极低 | 运营商物联网、大规模设备管理 |
| HTTP | TCP | 大(几百字节) | 请求/响应 | 高 | 网关/边缘设备、资源充足的设备 |

## 3.5 IoT网关与边缘计算

### IoT网关的核心功能

**协议转换**
Modbus/OPC-UA→MQTT

**数据预处理**
过滤/聚合/格式转换

**本地缓存**
断网续传/数据缓冲

**安全加密**
TLS/设备认证/防火墙

*网关是IoT系统的"翻译官"和"守门员"，连接感知层与平台层*

### 边缘计算架构

云端 Cloud：大数据分析/AI模型训练/长期存储

↓

边缘 Edge：实时处理/低延迟响应/本地决策

↓

端侧 Device：传感器/执行器/数据采集

### 边缘计算的价值与主流框架

**边缘计算的价值：**

1. 低延迟：毫秒级响应，适合实时控制场景
2. 减少带宽：本地过滤聚合，只传有价值数据
3. 数据隐私：敏感数据本地处理，不上传云端
4. 离线可用：断网时本地自治运行
5. 降低成本：减少云端存储和计算资源消耗

**主流边缘计算框架：**

1. Eclipse Kura：Java开源边缘网关框架
2. EdgeX Foundry：Linux基金会边缘中间件
3. AWS Greengrass：亚马逊边缘计算平台
4. Azure IoT Edge：微软边缘运行时
5. KubeEdge：K8s原生边缘计算（云原生）

## 3.6 实战案例：智慧城市环境监测系统

*案例背景：某新一线城市部署智慧城市环境监测系统，覆盖全市5000+监测点，实时采集空气质量、噪声、水质、气象等数据*




采集指标：PM2.5/PM10/SO2/NO2/O3/CO(空气质量)、噪声分贝、pH/溶解氧(水质)、温湿度/风速/气压(气象)，每5分钟上报一次

### 关键技术方案

**通信方案**：NB-IoT低功耗广域网，设备电池续航5年+
**消息协议**：MQTT QoS 1，EMQX集群支持百万连接
**数据存储**：TDengine时序数据库，写入性能100万点/秒
**边缘计算**：区域网关本地异常检测，超标立即告警

### 项目成果与价值

**数据规模**：日均采集1.4亿条数据，年数据量500亿+
**实时性**：数据从采集到展示延迟小于10秒
**应用价值**：污染溯源、空气质量预报、应急响应
**社会效益**：环保决策数据支撑，市民健康预警服务

**技术亮点**：NB-IoT+MQTT+TDengine的经典IoT技术栈；边缘网关实现本地异常检测（PM2.5超标立即触发告警），云端做长期趋势分析和AI预测模型训练；时序数据库相比传统关系型数据库写入性能提升10倍，存储压缩比达15:1。

## 3.7 IoT数据采集平台与技术选型

### 主流IoT数据采集平台对比

| 平台 | 厂商 | 核心能力 | 支持协议 | 适用场景 |
| --- | --- | --- | --- | --- |
| 阿里云IoT | 阿里巴巴 | 设备管理/规则引擎/数据分析 | MQTT/CoAP/HTTP/LoRa | 国内企业级IoT应用开发 |
| 华为云IoT | 华为 | 设备接入/设备管理/数据转发 | MQTT/CoAP/LwM2M/MQTT-SN | 运营商/智慧城市/工业IoT |
| AWS IoT Core | 亚马逊 | 设备网关/规则引擎/影子设备 | MQTT/HTTPS/WebSocket/LoRa | 全球部署/海外业务 |
| EMQX | 开源/EMQ | MQTT Broker/规则引擎/百万连接 | MQTT/MQTT-SN/CoAP/LwM2M | 自建IoT平台/高并发消息 |
| ThingsBoard | 开源 | 设备管理/数据可视化/告警 | MQTT/CoAP/HTTP/LwM2M | 开源IoT平台/快速原型验证 |

### 时序数据库选型

InfluxDB：最流行的开源时序DB，TICK技术栈，写入查询性能优秀
TDengine：国产高性能时序DB，100万点/秒写入，压缩比15:1
TimescaleDB：基于PostgreSQL，SQL兼容，适合复杂查询
OpenTSDB：基于HBase，适合超大规模监控数据

### 技术选型建议

小规模：EMQX+InfluxDB+Grafana，快速搭建
中大规模：EMQX集群+Kafka+TDengine+Flink
企业级：阿里云/华为云IoT平台，省心运维
边缘场景：EdgeX+KubeEdge，云边协同



# 大数据获取与采集

CONTENTS


01 分布式爬虫
Distributed Web Crawler

02 日志采集
Log Collection

03 物联网数据采集
IoT Data Collection

04 扩展的数据采集
Extended Data Collection

# 4 扩展的数据采集

## 4.1 API数据采集

### API数据采集概述

API（Application Programming Interface，应用程序编程接口）是系统间数据交换的标准化接口。通过API采集数据是最规范、最稳定的数据获取方式。

**优势**：数据结构化、更新及时、合规性好、无需解析HTML

**类型**：RESTful API、GraphQL、WebSocket、gRPC

### 常见公开API数据源

**开放数据平台**：国家统计局、天气API、高德地图

**社交媒体**：微博开放平台、Twitter API、抖音开放平台

**金融数据**：Tushare、聚宽、东方财富API

**AI能力**：OpenAI、百度AI、阿里云AI接口

### API采集关键技术

**认证机制**
API Key / OAuth 2.0
JWT Token / 签名认证

**分页拉取**
Offset/Limit分页
Cursor游标分页

**限流处理**
Rate Limit退避重试
令牌桶/漏桶算法

**增量同步**
时间戳/版本号
Webhook推送

### Python API采集代码示例（requests）

```python
# 分页拉取+限流+重试
import requests, time
def fetch_all_data(api_url, api_key):
headers = {"Authorization": f"Bearer {api_key}"}
all_data, page, cursor = [], 1, None
while True:
params = {"cursor": cursor, "limit": 100}
resp = requests.get(api_url, headers=headers, params=params, timeout=30)
if resp.status_code == 429: # 限流退避
time.sleep(int(resp.headers.get("Retry-After", 5))); continue
data = resp.json(); all_data.extend(data["items"])
cursor = data.get("next_cursor"); if not cursor: break
return all_data
```

重庆大学 CHONGQING UNIVERSITY

## 4.2 传感器与RFID数据采集

### 传感器数据采集

- **环境传感器**：温湿度/气压/光照/空气质量
- **工业传感器**：压力/流量/振动/温度
- **生物传感器**：心率/血压/血氧/血糖
- **位置传感器**：GPS/北斗/加速度/陀螺仪

*采集方式：ADC模数转换 → 单片机/MCU → 串口/I2C/SPI → 网关上传*

### RFID射频识别采集




- **RFID分类**：低频(LF 125KHz)/高频(HF 13.56MHz)/超高频(UHF 860-960MHz)/微波(2.4GHz)
- **读取距离**：无源标签几厘米到几米，有源标签可达几十米
- **应用场景**：物流追踪、仓储管理、门禁考勤、供应链、零售结算
- **标准协议**：ISO 14443(M1卡)、ISO 15693、EPC Gen2(UHF)

### 传感器与RFID采集对比

| 对比维度 | 传感器采集 | RFID采集 |
|---|---|---|
| 数据类型 | 连续模拟量（温度、压力等时序数据） | 离散事件（标签ID、读取时间、位置） |
| 采集频率 | 高频（毫秒到秒级，持续采集） | 事件驱动（标签进入读取范围时触发） |
| 数据量 | 大（持续产生，TB级/天） | 小（每次仅ID+时间戳，GB级/天） |
| 典型应用 | 环境监测、工业监控、可穿戴设备 | 物流追踪、仓储管理、门禁考勤 |

## 4.3 社交媒体数据采集

*社交媒体数据是舆情分析、用户画像、市场研究的重要数据源，具有实时性强、情感丰富、传播速度快等特点*

### 采集方式

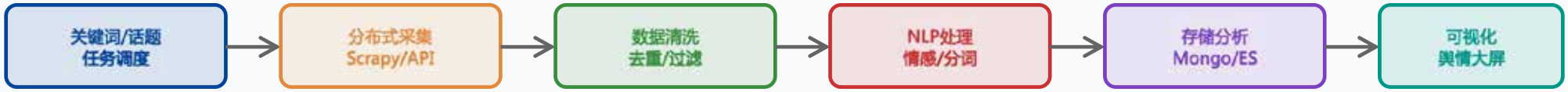
**官方API**
微博开放平台
Twitter API v2
抖音开放平台
合规稳定，有调用限额

**爬虫采集**
移动端API逆向
网页端数据抓取
数据量大，需反爬
注意合规风险

### 采集数据维度

**内容数据**：微博正文、评论、转发、点赞数、话题标签
**用户数据**：昵称、头像、粉丝数、关注数、注册时间、地域
**关系数据**：关注关系、转发链路、互动网络
**时空数据**：发布时间、GPS位置、签到地点

### 社交媒体采集技术架构




**合规与伦理注意事项**：社交媒体数据采集必须遵守平台API使用条款，不得绕过技术保护措施；不得采集用户隐私信息（手机号、身份证等）；遵守《个人信息保护法》，用户数据需匿名化处理；爬虫采集需控制频率，不得影响平台正常运行；数据仅用于研究或授权商业用途，不得转卖或用于非法目的。

## 4.4 数据质量治理

*数据质量是大数据分析的生命线——"Garbage In, Garbage Out"，采集阶段必须建立数据质量保障机制*

### 数据质量六大维度

| 准确性 | 完整性 | 一致性 | 时效性 | 唯一性 | 规范性 |
|---|---|---|---|---|---|
| Accuracy | Completeness | Consistency | Timeliness | Uniqueness | Validity |
| 数据真实反映客观事实 | 无缺失值和缺失记录 | 多源数据无矛盾冲突 | 数据及时更新可用 | 无重复记录和主键 | 符合格式和值域约束 |

### 采集阶段质量控制技术

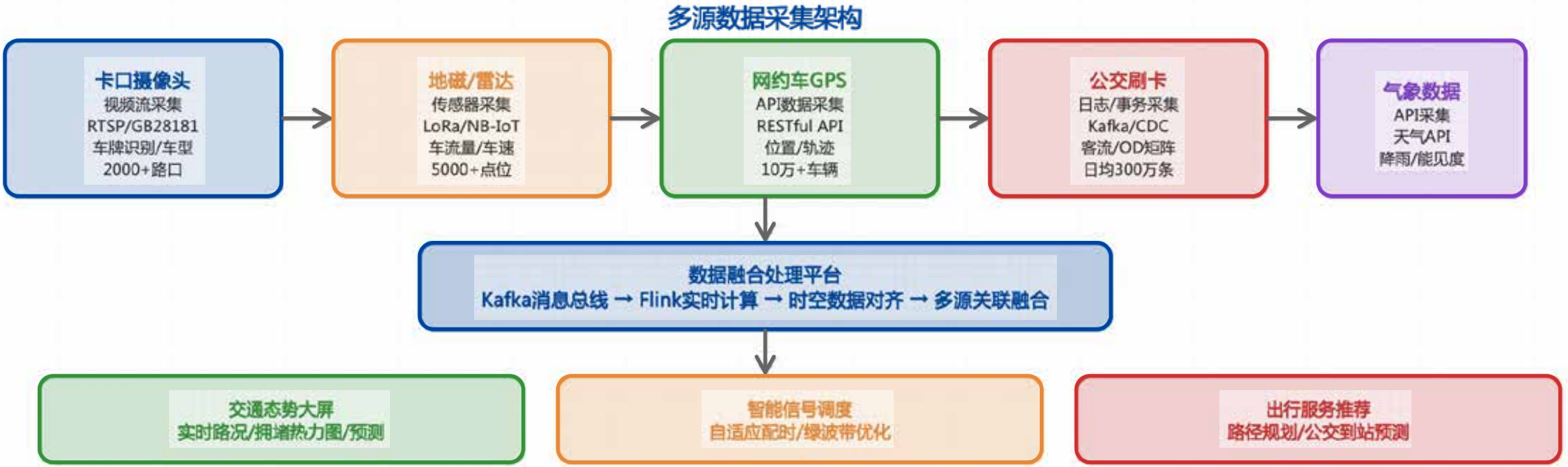
1. **数据校验规则**
   字段非空检查、格式校验（正则）、值域范围检查、类型转换校验、主键唯一性校验
2. **实时监控告警**
   采集速率监控、数据量异常检测、字段缺失率监控、延迟监控、失败率监控
3. **数据清洗策略**
   去重（MD5/SimHash）、缺失值填充（均值/中位数/前向填充）、异常值检测（3σ/IQR）、格式标准化
4. **数据血缘追踪**
   记录数据来源、采集时间、处理过程，支持溯源和问题定位

### 主流数据质量工具

| 工具 | 类型 | 特点 |
|---|---|---|
| Great Expectations | Python数据校验 | 声明式规则，自动生成文档 |
| Apache Griffin | 大数据质量平台 | 支持批/流，Hadoop生态 |
| DataCleaner | 数据清洗工具 | 可视化配置，丰富组件 |
| Deequ | Spark数据质量 | Amazon开源，Spark原生 |
| Informatica DQ | 企业级平台 | 商业产品，功能全面 |

## 4.5 综合案例：智慧交通多源数据融合采集

*某城市智慧交通项目融合5类数据源，实现交通态势实时感知与智能调度*




**项目成效**：融合5类数据源后，交通状态识别准确率从78%提升至95%；信号配时优化使主干道平均通行时间缩短23%；公交到站预测误差小于2分钟；日均处理数据量超过5TB，端到端延迟低于3秒。

**关键技术**：多源时空数据对齐（时间戳同步+空间网格编码）、数据质量治理（异常检测+缺失插值）、实时流处理（Flink+Ceph存储）、AI模型融合（多模态特征工程）。